

EXECUTIVE OS v4.0

OPERATIONAL SYSTEMS

BLUEPRINT

12 Core Operational Components — Developer Implementation Specification

*Billing & Subscriptions • User Authentication • Admin Dashboard • Quality Evaluation Prompt
Versioning • Legal Compliance • Disaster Recovery • LLM Failover Abuse Prevention • Multi-Channel
Sync • Notification Engine • Product Analytics*

February 2026 • CONFIDENTIAL

Table of Contents

1. Overview & Integration Strategy
2. Phase Mapping — Where Each Component Fits
3. Component 1: Billing & Subscription Infrastructure
4. Component 2: User Authentication & Account Management
5. Component 3: Admin Dashboard & Back-Office
6. Component 4: Conversation Evaluation & Quality Monitoring
7. Component 5: Prompt Versioning & A/B Testing
8. Component 6: Legal Compliance — GDPR, CCPA, Data Retention
9. Component 7: Disaster Recovery & Business Continuity
10. Component 8: LLM Provider Outage & Degraded Mode
11. Component 9: Abuse Prevention & Content Safety
12. Component 10: Multi-Channel State Sync
13. Component 11: Notification & Proactive Messaging Engine
14. Component 12: Product Analytics & Metrics
15. Cross-Component Dependencies
16. Database Schema Additions — Complete
17. Sprint Integration — Full Mapping
18. Appendix A: Environment Variables
19. Appendix B: Pydantic Contracts — All Components

1. Overview & Integration Strategy

The Executive OS v4.0 Blueprint, MCP Integration Specification, and MCP Server Deployment Plans (Waves 1–6) together define the agent's intelligence, tool connectivity, and server fleet. But they assume the existence of operational infrastructure that is never specified: how users sign up and pay, how the team monitors quality, how the system recovers from failure, and how legal obligations are met.

This document specifies the 12 operational components required to take Executive OS from a functional AI agent to a production-ready, revenue-generating, legally compliant product. Each component is defined at developer-grade depth: database schemas, API endpoints, implementation patterns, and sprint-level mapping into the existing 12-month build plan.

PRINCIPLE: These components integrate into the existing Blueprint timeline. They do not create a separate build track. Every component maps to specific Blueprint phases and sprints, sharing infrastructure and database tables where possible.

1.1 Document Relationship

Document	What It Defines	This Blueprint's Relationship
Executive OS v4.0 Blueprint	Core agent architecture, intelligence, tool deployment	This document layers INTO those sprints
MCP Integration Specification	MCP client infrastructure	This document shares database, auth, observability
MCP Deployment Plan (Waves 1–4)	30 launch servers	Billing enables monetization of these servers
MCP Wave 5–6 Expansion	10 post-launch servers	Analytics measures adoption of these servers
Operational Systems Blueprint (this)	12 operational components	The connective tissue that makes everything production-ready

1.2 Priority Classification

Priority	Components	Why
Launch-Blocking	Billing, User Auth, Legal Compliance, Disaster Recovery	Cannot ship without revenue, identity, legal protection, or data
Week-1 Critical	Admin Dashboard, Abuse Prevention, LLM Failover, Multi-Channel Sync	First customer experience; expose these gaps immediately
Month-1 Quality	Eval & Quality, Prompt Versioning, Notification Engine, Analytics	Operational excellence; detectable only in retrospect

2. Phase Mapping — Where Each Component Fits

Each of the 12 components maps to a specific phase in the Blueprint's 12-month plan. Components are built alongside existing Blueprint work, not after it.

Component	Phase	Months	Depends On	Blocks
1. Billing	Phase 1	Month 2–3	User Auth	Launch, Monetization
2. User Auth	Phase 1	Month 1–2	Database schema	Everything — all other components
3. Admin Dashboard	Phase 2	Month 5–6	User Auth, Billing, Observability	Operations, Support
4. Eval & Quality	Phase 2	Month 6–7	Brain Plane, Context Compiler	Prompt Versioning, A/B Testing
5. Prompt Versioning	Phase 2	Month 7	Eval & Quality, LLM Router	Safe prompt iteration
6. Legal Compliance	Phase 1	Month 2–4	User Auth, Database schema	Launch (legal requirement)
7. Disaster Recovery	Phase 1	Month 1–3	Database, S3, Redis	Launch (data safety)
8. LLM Failover	Phase 2	Month 4–5	LLM Router, Observability	Reliability SLA
9. Abuse Prevention	Phase 2	Month 5–6	Gateway Plane, User Auth, Billing	Content safety, Cost control
10. Multi-Channel Sync	Phase 1–2	Month 3–4	User Auth, Gateway Plane	WhatsApp + iMessage support
11. Notification Engine	Phase 2–3	Month 6–8	Multi-Channel, Knowledge files, Reactive Agent	Proactive Agent behavior
12. Product Analytics	Phase 2	Month 5–7	User Auth, Billing, Observability	Product decisions, Retention

LAUNCH-BLOCKING COMPONENTS

3. Component 1: Billing & Subscription Infrastructure

Without billing, you have no business — just a free tool burning money. This component handles plan management, payment processing, usage metering, and revenue operations.

3.1 Architecture

Billing lives in the Gateway Plane as a middleware layer. Every inbound message passes through a billing check before reaching the Brain Plane. The billing system integrates with Stripe Billing for payment processing and subscription management.

Data flow: Inbound message → Gateway Plane → Billing Middleware (check subscription active, check usage limits) → if valid: route to Brain Plane → after response: record usage → Stripe webhook handler updates subscription state.

3.2 Plan Structure

Plan	Price	Messages/Day	MCP Servers	LLM Tier	Features
Free Trial	\$0 (14 days)	20	3 (Calendar, Tasks, Search)	GPT-4o-mini only	Basic workflows, no proactive
Personal	\$19.99/mo	200	All Wave 1–4 (30)	Full multi-provider	Proactive briefings, all knowledge files
Professional	\$49.99/mo	Unlimited	All Waves 1–6 (40)	Full + priority routing	Research Engine, financial integrations
Enterprise	Custom	Unlimited	All + custom servers	Full + dedicated capacity	SSO, team features, SLA, admin console

3.3 Database Schema

TABLE: subscriptions

Column	Type	Description
id	UUID PK	Subscription identifier
user_id	UUID FK → users	Owner
stripe_subscription_id	TEXT UNIQUE	Stripe subscription ID
stripe_customer_id	TEXT	Stripe customer ID
plan	ENUM (free_trial, personal, professional, enterprise)	Subscription plan
status	ENUM (trialing, active, past_due, canceled, paused)	Subscription state
current_period_start	TIMESTAMPTZ	Billing period start
current_period_end	TIMESTAMPTZ	Billing period end
trial_end	TIMESTAMPTZ NULL	Trial expiration date

Column	Type	Description
cancel_at_period_end	BOOLEAN DEFAULT FALSE	Scheduled cancellation
daily_message_count	INT DEFAULT 0	Messages used today (reset at midnight UTC)
daily_message_limit	INT	Max messages per day for plan
monthly_mcp_cost_cents	INT DEFAULT 0	Accumulated MCP costs this billing period
monthly_mcp_budget_cents	INT	Max MCP cost per billing period
metadata	JSONB DEFAULT '{}'	Plan-specific config overrides
created_at	TIMESTAMPTZ	Creation timestamp
updated_at	TIMESTAMPTZ	Last update

TABLE: invoices

Column	Type	Description
id	UUID PK	Invoice identifier
user_id	UUID FK → users	Owner
stripe_invoice_id	TEXT UNIQUE	Stripe invoice ID
amount_cents	INT	Total amount in cents
status	ENUM (draft, open, paid, void, uncollectible)	Invoice state
period_start	TIMESTAMPTZ	Billing period start
period_end	TIMESTAMPTZ	Billing period end
pdf_url	TEXT NULL	Stripe-hosted invoice PDF
created_at	TIMESTAMPTZ	Creation timestamp

3.4 API Endpoints

Method	Path	Description	Auth
POST	/api/v1/billing/checkout	Create Stripe Checkout session for new subscription	User JWT
POST	/api/v1/billing/portal	Create Stripe Customer Portal session (manage plan)	User JWT
GET	/api/v1/billing/subscription	Get current subscription details + usage	User JWT
POST	/api/v1/billing/webhooks/stripe	Stripe webhook handler (subscription events)	Stripe signature
GET	/api/v1/billing/invoices	List user's invoices	User JWT
POST	/api/v1/billing/usage/record	Record a billable event (internal only)	Service token

3.5 Billing Middleware Logic

```

async def billing_middleware(message: InboundMessage) -> BillingDecision:
    sub = await get_subscription(message.user_id)
    if sub.status in ('canceled', 'paused'):
        return BillingDecision.BLOCKED_INACTIVE

```

```

if sub.status == 'past_due' and days_past_due(sub) > 7:
    return BillingDecision.BLOCKED_PAYMENT
if sub.daily_message_count >= sub.daily_message_limit:
    return BillingDecision.RATE_LIMITED
if sub.monthly_mcp_cost_cents >= sub.monthly_mcp_budget_cents:
    return BillingDecision.MCP_BUDGET_EXCEEDED
await increment_daily_count(message.user_id)
return BillingDecision.ALLOWED

```

3.6 Stripe Webhook Events

Event	Action
customer.subscription.created	Create subscription record, set plan limits
customer.subscription.updated	Update plan, status, period dates, limits
customer.subscription.deleted	Set status=canceled, schedule data retention per Section 8
invoice.paid	Update invoice record, reset monthly MCP cost counter
invoice.payment_failed	Set status=past_due, send payment failure notification to user
customer.subscription.trial_will_end	Send trial ending notification (3 days before)

3.7 Implementation Notes

- Use Stripe Checkout for initial signup — hosted payment page, PCI compliant, no card handling on your side.
- Use Stripe Customer Portal for plan changes, cancellations, payment method updates — again hosted by Stripe.
- Daily message counter resets via a cron job at midnight UTC. Use Redis INCR with TTL for real-time counting, persist to Postgres hourly.
- Dunning (failed payment retry): Stripe handles automatic retries. After 3 failures over 2 weeks, Stripe sets status to 'unpaid'. Your webhook handler sets status to 'canceled' and sends a final notification.
- Free trial: 14 days, no credit card required. At trial end, user is prompted to subscribe. If no action, account enters 'paused' state — data retained for 90 days, then deletion pipeline runs (Component 6).

4. Component 2: User Authentication & Account Management

Every other component depends on knowing who the user is. This component handles identity, sign-up, cross-channel linking, and account lifecycle.

4.1 Architecture

User authentication uses a hybrid model: WhatsApp and iMessage identify users by phone number (channel-native auth), while the web dashboard and API use JWT tokens issued by an identity provider. The canonical identity is the `user_id` in the `users` table; channel identifiers are linked via the `user_channels` table.

4.2 Identity Provider

Recommended: Clerk or Auth0. Both support phone-based auth (SMS OTP), social login (Google, Apple), and JWT issuance. Clerk is lighter-weight and cheaper for startups. Auth0 is more enterprise-ready. Either works. The system must support:

- Phone number as primary identifier (for WhatsApp/iMessage users)
- Email + password as secondary (for web dashboard access)
- Social login via Google and Apple (optional, for convenience)
- SMS OTP for phone verification during channel linking
- JWT issuance for API and dashboard authentication

4.3 Database Schema

TABLE: users

Column	Type	Description
id	UUID PK	Canonical user identifier — referenced by all other tables
external_auth_id	TEXT UNIQUE	ID from identity provider (Clerk/Auth0)
email	TEXT UNIQUE NULL	Email (optional for phone-only users)
phone_number	TEXT UNIQUE NULL	E.164 format phone number
display_name	TEXT	User's preferred display name
avatar_url	TEXT NULL	Profile picture URL
timezone	TEXT DEFAULT 'America/New_York'	User's timezone (for notifications, briefings)
locale	TEXT DEFAULT 'en-US'	Language/locale preference
onboarding_completed	BOOLEAN DEFAULT FALSE	Has completed the onboarding flow
account_status	ENUM (active, paused, suspended, deleted)	Account lifecycle state
created_at	TIMESTAMPTZ	Account creation
updated_at	TIMESTAMPTZ	Last profile update

Column	Type	Description
last_active_at	TIMESTAMPTZ	Last message sent or action taken
deletion_requested_at	TIMESTAMPTZ NULL	GDPR/CCPA deletion request timestamp

TABLE: user_channels

Column	Type	Description
id	UUID PK	Channel link identifier
user_id	UUID FK → users	Canonical user
channel	ENUM (whatsapp, imessage, webchat)	Channel type
channel_identifier	TEXT	Phone number, Apple ID, or session token
is_primary	BOOLEAN DEFAULT FALSE	Primary channel for proactive messages
verified	BOOLEAN DEFAULT FALSE	Phone/identity verified via OTP
linked_at	TIMESTAMPTZ	When this channel was linked
last_message_at	TIMESTAMPTZ NULL	Last message on this channel

4.4 Channel Linking Flow

When a user first messages on WhatsApp, the system checks if the phone number exists in user_channels. If not, a new user record is created with that phone number as the primary channel. Later, if the same user wants to also use iMessage or the web dashboard:

- **WhatsApp → iMessage linking:** User says 'I also want to use iMessage.' Agent sends an SMS OTP to the phone number on file. User provides the iMessage-associated phone/Apple ID. System verifies and creates a new user_channels row.
- **Any channel → Web dashboard:** User visits the web dashboard and logs in with phone number (SMS OTP) or email. System matches to existing user via phone number or creates a link.
- **Conflict resolution:** If a phone number is already linked to a different user_id, the system flags it for manual review rather than auto-merging (prevents account takeover).

4.5 API Endpoints

Method	Path	Description
POST	/api/v1/auth/signup	Create account (phone or email)
POST	/api/v1/auth/login	Authenticate and receive JWT
POST	/api/v1/auth/verify-otp	Verify SMS OTP for phone auth
POST	/api/v1/auth/link-channel	Link additional channel to account
GET	/api/v1/auth/me	Get current user profile
PUT	/api/v1/auth/me	Update profile (name, timezone, locale)
DELETE	/api/v1/auth/me	Request account deletion (triggers GDPR pipeline)
GET	/api/v1/auth/channels	List linked channels
DELETE	/api/v1/auth/channels/:id	Unlink a channel

5. Component 3: Admin Dashboard & Back-Office

You need a way to operate your own system. Without this, you're blind to user issues, cost overruns, and quality problems the moment you launch.

5.1 Dashboard Sections

Section	Key Metrics / Features	Data Source
System Health	API latency p50/p95/p99, error rate, uptime, active EC2 instances	CloudWatch
User Management	Search users, view profiles, subscription status, connection logs	PostgreSQL (users, subscriptions, history (read-only))
MCP Server Fleet	30–40 server health cards, connection status, latency, Redis rate, cost per server	Redis, MCP_servers table
Conversation Review	Flagged conversations, low-confidence responses, user-reported issues, quality scores	conversations, quality_scores tables
Financial Dashboard	MRR, churn rate, ARPU, cost per user, MCP cost breakdown	Stripe, API ML subscriptions, tool_executions
Content Moderation	Queue of flagged messages (abuse, safety), auto-flagged by classifier	moderation_queue table
Prompt Management	Active prompt versions, A/B test results, rollout status, rollout controls	prompt_versions table
Support Escalation	User support requests, linked to conversation context, assignment tracking	support_tickets table

5.2 Technology Stack

- **Frontend:** Next.js + Tailwind CSS. Deployed on Vercel or as a static site on S3/CloudFront. Internal-only (not user-facing).
- **API:** Extends the existing FastAPI backend with /api/v1/admin/* endpoints. Protected by admin JWT role.
- **Auth:** Admin users are a separate role in the identity provider. Enforced via JWT claims (role: admin). No shared auth with end users.
- **Real-time:** WebSocket connection for live error log stream, MCP server health updates, and moderation queue notifications.

5.3 Key API Endpoints

Method	Path	Description
GET	/api/v1/admin/users	List/search users with filters
GET	/api/v1/admin/users/:id	Full user detail (profile, subscription, channels, MCP connections)
GET	/api/v1/admin/users/:id/conversations	User's conversation history (read-only)
POST	/api/v1/admin/users/:id/suspend	Suspend account (abuse)
GET	/api/v1/admin/mcp/health	All MCP server health status
GET	/api/v1/admin/moderation/queue	Pending moderation items
POST	/api/v1/admin/moderation/:id/resolve	Resolve moderation item
GET	/api/v1/admin/metrics/financial	Revenue, cost, margin metrics
GET	/api/v1/admin/metrics/usage	DAU, MAU, message volume, feature adoption
GET	/api/v1/admin/prompts	List prompt versions and A/B test status
POST	/api/v1/admin/prompts/:id/rollback	Rollback to previous prompt version

WEEK-1 CRITICAL COMPONENTS

6. Component 4: Conversation Evaluation & Quality Monitoring

You need to know if the agent is working well. Without systematic evaluation, quality degrades silently in production with no warning until users churn.

6.1 Three-Layer Evaluation System

Layer	What It Measures	When It Runs	Implementation
Golden Test Suite	Baseline capability: does the agent handle 100+ representative prompts accurately?	On dev, 100 prompts daily	Pre-deployed behavior (not exact text)
Live Quality Scoring	Real-time quality of production responses	On every production response	LLM-based (e.g., Claude) scores 1–5 on 4 dimensions
User Feedback Loop	User satisfaction: did the response actually help?	On every response (optional)	Feedback stored, analyzed weekly. Negative feedback triggers review.

6.2 Database Schema

TABLE: eval_results

Column	Type	Description
id	UUID PK	Evaluation record
conversation_id	UUID FK	Conversation evaluated
message_id	UUID FK	Specific message evaluated
eval_type	ENUM (golden_test, live_sample)	How this eval was triggered
coherence_score	FLOAT (1.0–5.0)	Response coherence and clarity
helpfulness_score	FLOAT (1.0–5.0)	Did it address the user's need?
safety_score	FLOAT (1.0–5.0)	Content safety and appropriateness
tool_usage_score	FLOAT (1.0–5.0) NULL	Were tools used correctly? (NULL if no tools used)
overall_score	FLOAT (1.0–5.0)	Weighted average
evaluator_model	TEXT	Which LLM scored this (e.g., claude-haiku-4.5)
evaluator_rationale	TEXT NULL	LLM's reasoning for the scores
prompt_version_id	UUID FK NULL	Which prompt version generated this response
created_at	TIMESTAMPTZ	Evaluation timestamp

TABLE: user_feedback

Column	Type	Description
id	UUID PK	Feedback record

Column	Type	Description
user_id	UUID FK → users	Who gave feedback
message_id	UUID FK	Message rated
rating	ENUM (positive, negative)	Thumbs up or down
comment	TEXT NULL	Optional text feedback
created_at	TIMESTAMPTZ	Feedback timestamp

6.3 Golden Test Suite Structure

Each test case defines a user query, required context (connected MCP servers, user profile), expected behavior (not exact text), and pass/fail criteria:

```
{ "id": "GT-042",
  "category": "calendar_management",
  "query": "What do I have tomorrow morning?",
  "context": { "connected_servers": ["google-calendar-mcp"] },
  "expected_behavior": [
    "Calls list_events with tomorrow date range",
    "Returns events in chronological order",
    "Mentions meeting titles and times" ],
  "failure_conditions": [
    "Hallucinates events not in calendar",
    "Fails to call any tool",
    "Responds with generic answer instead of real data" ] }
```

6.4 Quality Alerts

Alert	Trigger	Action
Quality Drop	Average overall_score drops below 3.5 over 7 days	PagerDuty alert to on-call + auto-pause any active A/B test
Safety Violation	Any safety_score below 2.0	Conversation flagged in moderation queue + PagerDuty
Tool Misuse	tool_usage_score below 2.0 for 3+ consecutive messages	Flag conversation + review tool schemas for ambiguity
Negative Feedback Spike	Negative feedback rate exceeds 15% over 1-hour window	Alert to product team + snapshot prompt version and model config

7. Component 5: Prompt Versioning & A/B Testing

Prompts are the soul of the agent. Changing them without version control, staged rollouts, and rollback capability is reckless.

7.1 What Gets Versioned

- System prompt (the master prompt sent to LLM on every request)
- SOUL.md, IDENTITY.md, AGENTS.md templates (knowledge file structure)
- Context Compiler prompt templates (how context is assembled for the LLM)
- Tool description text (how tools are described in the LLM's tool schemas)
- Evaluator prompts (the prompts used by the eval system in Component 4)

7.2 Database Schema

TABLE: prompt_versions

Column	Type	Description
id	UUID PK	Version identifier
prompt_type	ENUM (system, soul_template, identity_template, agents_template, context_compiler, tool_description, eval)	Which prompt
version_number	INT	Sequential version (v1, v2, v3...)
content	TEXT	The full prompt text
change_description	TEXT	What changed and why
author	TEXT	Who made the change
status	ENUM (draft, canary, rolling_out, active, rolled_back , archived)	Deployment state
rollout_percentage	INT DEFAULT 0	0–100: what % of traffic gets this version
quality_metrics	JSONB NULL	Aggregated eval scores while in canary/rollout
created_at	TIMESTAMPTZ	Creation timestamp
activated_at	TIMESTAMPTZ NULL	When it reached 100% rollout
rolled_back_at	TIMESTAMPTZ NULL	If rolled back, when

7.3 Rollout Pipeline

Stage 1 — Draft: New prompt version created. Not serving any traffic. Author can preview responses in a sandbox.

Stage 2 — Canary (5%): 5% of traffic routed to new version. Eval system (Component 4) scores both versions. Minimum 200 evaluations before proceeding.

Stage 3 — Rolling Out (25% → 50% → 100%): If canary quality metrics meet or exceed current version, gradually increase traffic. Each step requires 200+ evaluations and quality >= current version.

Stage 4 — Active: 100% traffic. Previous version archived.

Rollback: At any stage, if quality drops below threshold or safety violation occurs, immediately revert to previous active version. Rollback takes effect within 60 seconds (cache TTL).

7.4 LLM Router Integration

The LLM Router (Blueprint Section 11) already selects which provider to use. Extend it with a `prompt_version_selector` that runs before provider selection:

```
async def select_prompt_version(user_id: str, prompt_type: str) -> PromptVersion:
    active = await get_active_versions(prompt_type) # May be 1 or 2
    if len(active) == 1: return active[0]
    # A/B test: hash user_id to deterministically assign variant
    bucket = hash(user_id + active[1].id) % 100
    if bucket < active[1].rollout_percentage: return active[1] # New
    return active[0] # Current
```

Users always get the same variant for the duration of a rollout (deterministic hashing). This prevents confusing behavior where the agent's personality shifts between messages.

8. Component 6: Legal Compliance — GDPR, CCPA, Data Retention

You store deeply personal data: conversations, behavioral profiles, financial transactions, calendar events, emails, location data. Legal compliance is not optional.

8.1 Data Subject Rights

Right	Regulation	Implementation	SLA
Right to Access	GDPR Art. 15, CCPA	Data export pipeline: generate ZIP of all user data (GDPR) or knowledge files (CCPA)	30 days (GDPR), 45 days (CCPA)
Right to Portability	GDPR Art. 20	Export in machine-readable JSON format	30 days
Right to Erasure	GDPR Art. 17, CCPA	Cascade deletion pipeline across all tables, knowledge files, Redis caches, S3 objects, Stripe data	30 days
Right to Rectification	GDPR Art. 16	User can update profile, correct knowledge files in conversation	Immediate
Right to Restrict Processing	GDPR Art. 18	Pause account: retain data but stop processing	Immediate (set account_status=pended)
Right to Object (Marketing)	GDPR Art. 21	Opt-out of proactive messages and analytics	Immediate

8.2 Data Deletion Pipeline

When a user requests account deletion (DELETE /api/v1/auth/me), the following cascade executes:

- **Immediate (within 1 hour):** Set account_status=deleted. Stop all processing. Revoke all OAuth tokens. Remove from active user caches.
- **Within 24 hours:** Delete all knowledge files from S3 (SOUL.md, IDENTITY.md, AGENTS.md, TOOLS.md, CONTACTS.md, ROUTINES.md, PREFERENCES.md, HEARTBEAT.md, USER.md). Delete all conversation records. Delete all eval_results and user_feedback. Delete all tool_executions. Delete Redis keys (daily counters, rate limiters, MCP cost trackers).
- **Within 7 days:** Delete from Stripe (customer record, subscription, invoices). Delete from identity provider (Clerk/Auth0). Remove from database backups by marking for exclusion in next backup rotation.
- **Within 30 days:** Verify deletion across all systems. Generate deletion confirmation log. Notify user via email that deletion is complete.

8.3 Data Retention Policies

Data Type	Active User Retention	After Cancellation	After Deletion Request
Conversations	Indefinite	90 days then delete	24 hours
Knowledge files (S3)	Indefinite	90 days then delete	24 hours
Tool execution logs	12 months rolling	90 days then delete	24 hours
Eval results	6 months rolling	Anonymize (remove user_id)	Anonymize immediately
Billing records	7 years (tax law)	7 years (tax law)	7 years (legal hold, anonymized)
Access logs	90 days	90 days	90 days (anonymized)
OAuth tokens	Until revoked	Immediate revocation	Immediate revocation

8.4 Privacy Policy Requirements

Your privacy policy must explicitly document: what data is collected (enumerated by type), why (legal basis for each), how long it's retained, who it's shared with (LLM providers — name each one), how users exercise their rights, and your data protection contact. The privacy policy must be accessible via WhatsApp (user asks 'what data do you store about me?', agent provides link) and the web dashboard.

9. Component 7: Disaster Recovery & Business Continuity

Backups you haven't tested restoring are not backups. This component ensures the system can recover from any failure scenario.

9.1 Recovery Objectives

Metric	Target	What It Means
RTO (Recovery Time Objective)	30 minutes	System fully operational within 30 min of any failure
RPO (Recovery Point Objective)	5 minutes	Maximum 5 minutes of data loss in worst case
Backup Frequency	Continuous (WAL streaming) + daily snapshots	Continuous recovery to any second in last 7 days
Backup Retention	7 days continuous, 30 days daily snapshots	Can restore to any daily snapshot in last month
Restore Test Frequency	Monthly	Automated restore test to staging environment every month

9.2 Infrastructure Redundancy

Component	Redundancy Strategy	Failover Time
PostgreSQL (RDS)	Multi-AZ with read replica. WAL streaming to S3. Daily snapshots.	Automatic (Multi-AZ) or manual (Snapshot)
Redis (ElastiCache)	Multi-AZ with automatic failover. Redis Cluster for sharding.	< 30 sec
S3 (Knowledge Files)	Cross-region replication to secondary region. Versioning enabled.	N/A (S3 is 99.999999999% durable)
ECS/Fargate (Compute)	Multi-AZ task placement. Minimum 2 tasks per service.	< 60 sec (ECS replaces unhealthy tasks)
ALB (Load Balancer)	Multi-AZ by default. Health checks every 10 sec.	Automatic

9.3 Failure Runbooks

Scenario	Detection	Response	Recovery
Database down	RDS event notification + health checks	Multi-AZ failover. Alert on-call.	Verify replica promotion. Check connection pools. Test queries.
Bad deployment	Error rate > 5% within 5 min	Auto-deploy via CodeDeploy Blue/Green	Previous stable version restored. Post-mortem within 24h.
AWS regional outage	CloudWatch cross-region metrics	DR failover to secondary region (RDS, S3, etc.)	Restore from health checks. Replay WAL for RDS. Verify S3 consistency.
Knowledge file corruption	File versioning + integrity checks	Restore previous version from S3 versioning	Automated checksum validation on every read. Manual spot-checks.
Accidental data deletion	Audit log in CloudTrail	Restore from point-in-time backup (RDS PITR, S3 versioning)	Recover to state before deletion. Verify data integrity.

10. Component 8: LLM Provider Outage & Degraded Mode

The multi-provider LLM Router routes between Claude, GPT-4o, Gemini, and Llama 4. But failover behavior needs precise specification.

10.1 Provider Health Monitoring

Background health probe for each provider. Runs every 30 seconds. Sends a minimal test prompt and measures latency + success. Not per-request — this is a background loop separate from user traffic.

```
class LLMProviderHealth:
    provider: str # 'anthropic', 'openai', 'google', 'local'
    status: str # 'healthy', 'degraded', 'down'
    latency_p95_ms: float # Rolling 5-min p95 latency
    error_rate: float # Rolling 5-min error rate (0.0-1.0)
    last_success: datetime # Last successful health probe
    consecutive_failures: int # Consecutive failed probes
```

10.2 Failover Matrix

Condition	Detection	Action	User Impact
Single provider returns 500	Health probe failure (2 consecutive)	Mark provider 'degraded'. Route traffic to next priority provider	Minor service interruption
Single provider latency p95 exceeds threshold	Rolling p95 exceeds threshold	Reduce traffic to 25%. Route overflow to next priority	Slight latency increase for some requests
Primary + secondary down	Two providers marked 'down'	Route all traffic to remaining external providers	Possible quality reduction if fallback is weaker model
All external providers down	All 3 external providers 'down'	DEGRADED MODE: route all traffic to local Llama 4	Reduced quality. User sees: 'Running on reduced capacity'
All providers including local down	Complete infrastructure failure	MAINTENANCE MODE: return static message	User message: 'I'm temporarily unavailable. Back shortly'

10.3 Degraded Mode Behavior

When operating on local Llama 4 only:

- Tier 1 and Tier 2 tasks work normally (simple queries, single-tool tasks)
- Tier 3 tasks (multi-step, hierarchical) are queued for retry when external providers recover
- Research Engine is disabled (requires Claude Sonnet)
- Morning briefing generation is deferred
- MCP tool calls continue normally (tool execution doesn't depend on LLM provider)
- User notification: first message in degraded mode includes a one-line notice. Not repeated on every message.

10.4 Recovery

When a provider recovers (3 consecutive successful health probes), it's marked 'healthy' and gradually receives traffic: 10% → 25% → 50% → 100% over 5 minutes. This prevents a thundering herd on a freshly-recovered provider. Queued Tier 3 tasks are replayed in order.

11. Component 9: Abuse Prevention & Content Safety

Privilege isolation protects against malicious data. This component protects against malicious users.

11.1 Threat Model

Threat	Vector	Impact	Mitigation
Jailbreak attempts	Prompt injection via user messages	Agent ignores safety guidelines, provides harmful/unsafe responses	Input classifier + output classifier + safety_score in eval system
Cost abuse	Excessive API calls (500 DPMs per user per hour)	High usage costs	Per-user rate limit + daily cost cap + circuit breaker
Harassment via agent	Guiding agent to draft abusive messages	Legal/regulatory risk	Output content classifier + review queue
Bot/spam accounts	Automated signups to abuse infrastructure	Infrastructure costs, degraded service	Phone verification required + rate limiting + CAPTCHA on web signups
Data exfiltration	Tricking agent into revealing private user data	Privacy breach	RLS on all queries + user_id scoping + no cross-user data access

11.2 Rate Limiting Hierarchy

Level	Limit	Window	Enforcement
Gateway (per-user)	Plan limit (20/200/unlimited per day)	24 hours (midnight UTC time)	Billing middleware (Component 1)
Gateway (burst)	10 messages per minute	1 minute sliding window	Redis sliding window counter
MCP (per-server)	Server-specific (default 30/min)	1 minute sliding window	MCP Integration Spec Section 10.3
MCP (daily cost)	Plan-specific budget	Billing period	Redis daily counter + budget check
LLM (per-provider)	Provider rate limit	Per provider specs	LLM Router backoff + failover

11.3 Content Safety Pipeline

Input classifier: Lightweight model (GPT-4o-mini or dedicated safety model) scores every inbound message for: prompt injection attempt, harassment/abuse content, self-harm signals, and illegal activity requests. Runs in parallel with Brain Plane processing (does not add latency). If flagged, the response is held for safety review before delivery.

Output classifier: Same model scores every outbound response before delivery. Checks for: harmful content generation, PII leakage (agent accidentally including another user's data), and policy violations. If flagged, response is replaced with a safe fallback and the conversation is queued for human review.

Circuit breaker: If a single user triggers 3+ safety flags in 1 hour, their account is temporarily rate-limited to 5 messages/hour and all responses go through output classifier synchronously (adding latency but ensuring safety). Admin notification sent.

11.4 Database Schema

TABLE: moderation_queue

Column	Type	Description
id	UUID PK	Moderation item

Column	Type	Description
user_id	UUID FK → users	Flagged user
message_id	UUID FK	Flagged message
flag_type	ENUM (input_safety, output_safety, user_reporting, jailbreak)	When triggered, the flag
severity	ENUM (low, medium, high, critical)	Severity classification
classifier_score	FLOAT	Confidence score from safety classifier
status	ENUM (pending, reviewing, resolved, escalated)	Review state
reviewer_id	UUID NULL	Admin who reviewed
resolution	ENUM (false_positive, warning_issued, content_removed, account_suspended) NULL	Content
created_at	TIMESTAMPTZ	Flag timestamp
resolved_at	TIMESTAMPTZ NULL	Resolution timestamp

12. Component 10: Multi-Channel State Sync

If a user starts a conversation on WhatsApp and continues on iMessage, the agent must know it's the same conversation.

12.1 Conversation Model

A conversation is a logical thread that spans channels. The user_channels table (Component 2) links channels to users. The conversations table links conversations to users, not channels. Each message records which channel it arrived on, but the Brain Plane always sees the full conversation regardless of channel.

TABLE: conversations (Blueprint Section 3 – extended)

Column (New/Modified)	Type	Description
active_channel	ENUM (whatsapp, imessage, web, api) and NULL	Channel of most recent message (for response routing)
channels_used	TEXT[] DEFAULT '{}'	Array of all channels used in this conversation

TABLE: messages (Blueprint Section 3 – extended)

Column (New/Modified)	Type	Description
channel	ENUM (whatsapp, imessage, web, api) and NULL	Which channel this message arrived on / was sent via
channel_message_id	TEXT NULL	Platform-specific message ID (WhatsApp msg ID, iMessage GUID)

12.2 Response Routing

When the Brain Plane generates a response, it must be delivered on the correct channel. The rule is simple: **respond on the channel the user last messaged from**. The Gateway Plane checks conversation.active_channel and routes accordingly. If the active channel is unavailable (e.g., WhatsApp API is down), queue the response and retry, rather than delivering on a different channel unexpectedly.

12.3 Simultaneous Channel Handling

If a user sends messages on two channels within a short window (< 30 seconds), the system must not create a race condition. Implementation: acquire a per-user distributed lock (Redis SETNX with 30s TTL) before processing any message. The second message waits for the first to complete. This ensures the conversation state is consistent.

12.4 Channel-Specific Formatting

Channel	Max Message Length	Rich Content Support	Formatting Constraints
WhatsApp	4,096 chars	Buttons (3 max), list messages (No text, images, bold, or text)	Monospace via <code>code</code> . Italic via <i>text</i> . Plain text only. Links auto-preview.
iMessage	20,000+ chars	No buttons. Images. Links with preview	Full markdown, code blocks, tables, images, tables, tables
Web Dashboard	Unlimited	Full markdown, code blocks, tables, images, tables, tables	Raw structured data.
API	Unlimited	JSON structured responses	

The Gateway Plane's response formatter must adapt the Brain Plane's output to each channel's constraints. For WhatsApp: truncate at 4,096 chars, convert markdown to WhatsApp formatting, convert lists to WhatsApp

list messages. For iMessage: strip all formatting, convert to plain text. The Brain Plane should never be channel-aware — formatting is a Gateway concern.

MONTH-1 QUALITY COMPONENTS

13. Component 11: Notification & Proactive Messaging Engine

The agent isn't just reactive. It sends morning briefings, task reminders, deadline alerts, and contextual nudges. This component specifies how, when, and where.

13.1 Notification Types

Type	Priority	Trigger	Example
Morning Briefing	Medium	Scheduled: user's preferred time (from ROUTINE Summary)	"Good morning! Here are 4 meetings today, starting at 9 AM..."
Deadline Alert	High	Calendar event approaching or "Deadline in 2 days. Want to work on it now?"	"Deadline in 2 days. Want to work on it now?"
Task Reminder	Medium	Todoist/Asana due date approaching or "You have 3 tasks due today. Top priority: finalize Q3 budget."	"You have 3 tasks due today. Top priority: finalize Q3 budget."
Event Change	High	Calendar event modified by another user or "Cancelled your 2 PM to 3 PM."	"Cancelled your 2 PM to 3 PM."
Tool Suggestion	Low	User behavior pattern + unconnected app or "You use Slack often. Want me to connect?"	"You use Slack often. Want me to connect?"
Financial Alert	High	Unusual transaction, budget threshold (Plaid/Stripe) or "Spent \$2,400 on your Amex. Want details?"	"Spent \$2,400 on your Amex. Want details?"
Travel Alert	Critical	Flight status change (Duffel) or "Your 6 PM flight to SF is delayed 45 min. New departure: 6:45 PM."	"Your 6 PM flight to SF is delayed 45 min. New departure: 6:45 PM."
Weekly Summary	Low	Scheduled: Sunday evening or "This week: 23 meetings, 8 tasks completed, 14 remaining..."	"This week: 23 meetings, 8 tasks completed, 14 remaining..."

13.2 Notification Scheduling

TABLE: scheduled_notifications

Column	Type	Description
id	UUID PK	Notification identifier
user_id	UUID FK → users	Recipient
type	ENUM (morning_briefing, deadline, task_reminder, event_change, tool_suggestion, financial, travel, weekly_summary)	Notification category
priority	ENUM (low, medium, high, critical)	Delivery priority
channel	ENUM (whatsapp, imessage, web, auto) DEFAULT 'auto' ('auto' = user's primary channel)	Delivery channel
scheduled_for	TIMESTAMPTZ	When to deliver (timezone-aware)
payload	JSONB	Notification content (template + variables)
status	ENUM (pending, delivered, failed, cancelled) DEFAULT 'pending'	Delivery status
suppression_reason	TEXT NULL	If suppressed: 'quiet_hours', 'rate_limit', 'user_opted_out'
delivered_at	TIMESTAMPTZ NULL	Actual delivery timestamp
created_at	TIMESTAMPTZ	Creation timestamp

13.3 Quiet Hours & Backpressure

- **Quiet hours:** Default 10 PM – 7 AM in user's timezone. Configurable via PREFERENCES.md. Critical priority (travel alerts, financial alerts) bypass quiet hours. All others are queued until quiet hours end.
- **Rate limiting:** Maximum 5 proactive messages per day. Maximum 2 per hour. Critical priority doesn't count toward limits.
- **Deduplication:** If the same notification type + entity (e.g., same calendar event) triggers multiple times within 1 hour, only the latest is delivered.
- **User absence:** If user hasn't read any messages in 3+ days, reduce to Critical-only notifications. After 7 days of inactivity, pause all proactive messages. Resume when user sends a message.

13.4 Timezone Handling

All scheduling is timezone-aware. The user's timezone is stored in `users.timezone` (Component 2). When the user travels (detected via calendar events in different timezones, or Tesla location changes, or explicit user statement), the notification engine temporarily adjusts:

- Morning briefing shifts to new timezone within 24 hours of detected travel
- Quiet hours shift immediately
- The agent acknowledges the timezone change: 'Noticed you're in London now. I'll adjust your briefing time accordingly.'

14. Component 12: Product Analytics & Metrics

System metrics tell you servers are up. Product metrics tell you users care. Without analytics, you can't make product decisions.

14.1 Core Metrics

Metric	Definition	Target	Data Source
DAU / WAU / MAU	Unique users who sent 1+ messages in a day	See goals on product page	messages table (distinct user_id by date)
D1 / D7 / D30 Retention	% of users who return after 1/7/30 days from last message sent	D1 > 40% / D7 > 25% / D30 > 15%	messages table (distinct user_id by date)
Messages per DAU	Average messages per active user per day	> 10	messages table count / DAU
Task Completion Rate	% of user requests the agent fully resolved (no follow-up needed)	> 75%	conversation analysis
MCP Server Adoption	% of users who connected each MCP server	> 60%	user_servers table
Tool Usage Distribution	Which MCP tools are invoked most frequently		tool_executions table
Response Quality (avg)	Average overall_score from evals	> 4.5 / 5.0	eval_results table
User Satisfaction (NPS)	Net Promoter Score from periodic surveys	> 50	user_feedback table + survey tool
MRR / Churn Rate	Monthly recurring revenue and monthly cancellations	> \$100k / < 5%	Stripe API + subscriptions table
ARPU	Average revenue per user	> \$15/mo blended	Stripe revenue / MAU
Cost per User	LLM + MCP + infrastructure cost per active user per month	< \$5	LLM logs + tool_executions + hosting
Onboarding Completion	% of new users who complete onboarding (connect to MCP)	> 70%	onboarding_completed + mcp_user_servers

14.2 Analytics Pipeline

Event collection: Every significant user action emits an analytics event to a lightweight event bus (AWS EventBridge or a simple SQS queue). Events include: message_sent, message_received, tool_invoked, mcp_server_connected, onboarding_step_completed, feedback_given, subscription_created, subscription_canceled.

Storage: Raw events → S3 (Parquet format, partitioned by date). Aggregated metrics → PostgreSQL (analytics_daily table with pre-computed DAU, MAU, retention, etc.). Real-time dashboards → Redis (current-day counters).

Visualization: Admin Dashboard (Component 3) displays product metrics. For deeper analysis, connect S3 Parquet files to a BI tool (Metabase, Preset, or Looker). Start with the Admin Dashboard; add BI tool when you need cohort analysis and funnels.

14.3 Database Schema

TABLE: analytics_events

Column	Type	Description
id	UUID PK	Event identifier
user_id	UUID FK → users	Who triggered the event

Column	Type	Description
event_type	TEXT	Event name (e.g., 'message_sent', 'tool_invoked')
properties	JSONB	Event-specific properties (tool_name, server_id, etc.)
channel	ENUM (whatsapp, imessage, Onchain, Email, etc.)	Channel type
session_id	UUID NULL	Conversation session (groups related events)
created_at	TIMESTAMPTZ	Event timestamp

TABLE: analytics_daily (pre-aggregated, materialized view or cron job)

Column	Type	Description
date	DATE PK	Calendar date
dau	INT	Unique active users
messages_sent	INT	Total user messages
messages_received	INT	Total agent responses
tool_invocations	INT	Total MCP + native tool calls
mcp_servers_connected	INT	New MCP server connections today
new_users	INT	New signups
churned_users	INT	Cancellations
mrr_cents	INT	Monthly recurring revenue snapshot
total_cost_cents	INT	Total platform cost (LLM + MCP + infra)
avg_quality_score	FLOAT	Average eval overall_score

15. Cross-Component Dependencies

The 12 components are interconnected. This dependency map shows which components must be built first and which can be parallelized.

Component	Hard Dependencies (must exist first)	Soft Dependencies (enhances but not required)
1. Billing	User Auth	Analytics (revenue metrics)
2. User Auth	Database schema	—
3. Admin Dashboard	User Auth, Billing, Observability	All other components (displays their data)
4. Eval & Quality	Brain Plane, Context Compiler	Prompt Versioning (versions to measure)
5. Prompt Versioning	Eval & Quality (to measure impact), LLM Router	A/B testing framework
6. Legal Compliance	User Auth, Database (for deletion cascade)	All data-producing components
7. Disaster Recovery	Database, S3, Redis (must exist to back up)	—
8. LLM Failover	LLM Router, Observability	Eval (to measure degraded quality)
9. Abuse Prevention	Gateway Plane, User Auth, Billing	Eval (safety_score), Moderation queue
10. Multi-Channel Sync	User Auth (channel linking), Gateway Plane	Notification Engine
11. Notification Engine	Multi-Channel Sync, Knowledge files	Analytics (delivery metrics)
12. Product Analytics	User Auth, Billing, Observability	All components (events to track)

16. Database Schema Additions — Complete

This document adds 8 new tables and extends 2 existing tables. Combined with the Blueprint's 19 tables and the MCP Spec's 2 tables, the total schema is 29 tables.

Table	Component	Purpose	Key Relationships
subscriptions	1. Billing	Plan, payment, usage limits	FK → users
invoices	1. Billing	Payment history	FK → users
users (extended)	2. User Auth	Canonical identity	Referenced by all tables
user_channels	2. User Auth + 10. Multi-Channel	Channel-to-user linking	FK → users
eval_results	4. Eval & Quality	Automated quality scores	FK → conversations, messages
user_feedback	4. Eval & Quality	User thumbs up/down	FK → users, messages
prompt_versions	5. Prompt Versioning	Prompt text + rollout state	Referenced by eval_results
moderation_queue	9. Abuse Prevention	Flagged content for review	FK → users, messages
scheduled_notifications	11. Notification Engine	Proactive message scheduling	FK → users
analytics_events	12. Analytics	Raw user action events	FK → users
analytics_daily	12. Analytics	Pre-aggregated daily metrics	Standalone (materialized)
conversations (extended)	10. Multi-Channel	Added: active_channel, channel_id	Existing table, 2 new columns
messages (extended)	10. Multi-Channel	Added: channel, channel_message_id	Existing table, 2 new columns

17. Sprint Integration — Full Mapping

Each component maps to specific sprints in the Blueprint's 12-month plan. Components are built alongside existing Blueprint work, not in a separate track.

Month	Blueprint Work	Operational Components (This Document)	Deliverables
1	Infrastructure, observability, User Authn	DB Schema + identity provider setup	Disaster Recovery (backups, configs, MDR) configured. RDR
2	Gateway Plane, WhatsApp, Stripe	API Integration, subscriptions table	Billing legal (privacy, policy, creative) working. Privacy policy
3	Native connectors, Tool Exchange	Multi-Contexts, conversation model, Cross-contexting	Legal (discovery, legal) on working. CPE, Expertise, pip
4	Brain Plane, ReAct engine, LLM	Failover (health probes, failover matrix)	Prograded health monitoring live. Automatic failover tested.
5	Behavioral Intelligence, privacy	Abuse Prevention (rate limiting, content classifiers)	Assess, pub classifiers, dep. Ad Ad Dashboard (user syst
6	Knowledge files, SOUL/IDEA	EVAL & Quality (golden test suite, live scoring)	100 feedback loop, live quality dashboard (12% in demo, 10% in
7	Profiling engine, HEARTBEAT	Prompt Versioning (version control, canary)	Pipeline // Pipeline Analytics (analytics dashboard, daily aggr
8-9	MCP client + Wave 1 server	All components operational — no new builds	Full operational and tuning alongside MCP deployment
10-12	Waves 2–4, onboarding UX	Evaluating prompt optimization via A/B tests	Analysis of operational infrastructure, legal review of E&P

18. Appendix A: Environment Variables

Variable	Component	Description
STRIPE_SECRET_KEY	Billing	Stripe API secret key
STRIPE_WEBHOOK_SECRET	Billing	Stripe webhook signing secret
STRIPE_PRICE_ID_PERSONAL	Billing	Stripe Price ID for Personal plan
STRIPE_PRICE_ID_PROFESSIONAL	Billing	Stripe Price ID for Professional plan
AUTH_PROVIDER	User Auth	Identity provider ('clerk' or 'auth0')
AUTH_SECRET_KEY	User Auth	Identity provider secret key
AUTH_ISSUER_URL	User Auth	JWT issuer URL for verification
DAILY_MSG_LIMIT_FREE	Billing	Daily message limit for free tier (default: 20)
DAILY_MSG_LIMIT_PERSONAL	Billing	Daily message limit for personal (default: 200)
EVAL_SAMPLE_RATE	Eval & Quality	% of responses to evaluate (default: 0.10)
EVAL_MODEL	Eval & Quality	Model for automated evaluation (default: claude-haiku-4.5)
SAFETY_CLASSIFIER_MODEL	Abuse Prevention	Model for content safety (default: gpt-4o-mini)
SAFETY_CIRCUIT_BREAKER_THRESHOLD	Abuse Prevention	Flags in 1hr before circuit breaker (default: 3)
NOTIFICATION_QUIET_START	Notifications	Default quiet hour start (default: 22:00)
NOTIFICATION_QUIET_END	Notifications	Default quiet hour end (default: 07:00)
NOTIFICATION_MAX_DAILY	Notifications	Max proactive messages per day (default: 5)
LLM_HEALTH_PROBE_INTERVAL_SECONDS	LLM Failover	Health probe interval (default: 30)
LLM_DEGRADED_MODE_ENABLED	LLM Failover	Allow fallback to local Llama 4 (default: true)
DATA_RETENTION_DAYS_CANCELED	Legal	Days to retain data after cancellation (default: 90)
ANALYTICS_EVENT_BUS	Analytics	Event bus ARN (EventBridge or SQS URL)

19. Appendix B: Pydantic Contracts — All Components

Key Pydantic models for cross-component contracts:

Billing

```
class Subscription(BaseModel):
    id: UUID
    user_id: UUID
    plan: Literal['free_trial', 'personal', 'professional', 'enterprise']
    status: Literal['trialing', 'active', 'past_due', 'canceled', 'paused']
    daily_message_limit: int
    daily_message_count: int = 0
    monthly_mcp_budget_cents: int
    monthly_mcp_cost_cents: int = 0
```

User Auth

```
class User(BaseModel):
    id: UUID
    email: Optional[str] = None
    phone_number: Optional[str] = None
    display_name: str
    timezone: str = 'America/New_York'
    account_status: Literal['active', 'paused', 'suspended', 'deleted']

class UserChannel(BaseModel):
    user_id: UUID
    channel: Literal['whatsapp', 'imessage', 'web', 'api']
    channel_identifier: str
    is_primary: bool = False
    verified: bool = False
```

Eval & Quality

```
class EvalResult(BaseModel):
    conversation_id: UUID
    message_id: UUID
    coherence_score: float # 1.0-5.0
    helpfulness_score: float
    safety_score: float
    tool_usage_score: Optional[float] = None
    overall_score: float
    prompt_version_id: Optional[UUID] = None
```

Prompt Versioning

```
class PromptVersion(BaseModel):
    id: UUID
    prompt_type: Literal['system', 'soul_template', 'identity_template',
                        'agents_template', 'context_compiler', 'tool_description', 'evaluator']
    version_number: int
```

```
content: str
status: Literal['draft', 'canary', 'rolling_out', 'active', 'rolled_back', 'archived']
rollout_percentage: int = 0
```

Notifications

```
class ScheduledNotification(BaseModel):
    user_id: UUID
    type: Literal['morning_briefing', 'deadline', 'task_reminder',
                 'event_change', 'tool_suggestion', 'financial', 'travel', 'weekly_summary']
    priority: Literal['low', 'medium', 'high', 'critical']
    channel: Literal['whatsapp', 'imessage', 'web', 'auto'] = 'auto'
    scheduled_for: datetime
    payload: dict
    status: Literal['pending', 'delivered', 'failed', 'canceled', 'suppressed']
```

LLM Failover

```
class LLMProviderHealth(BaseModel):
    provider: Literal['anthropic', 'openai', 'google', 'local']
    status: Literal['healthy', 'degraded', 'down']
    latency_p95_ms: float
    error_rate: float # 0.0-1.0
    last_success: datetime
    consecutive_failures: int = 0
```

— END OF OPERATIONAL SYSTEMS BLUEPRINT —

This document is a standalone addendum to the Executive OS v4.0 Blueprint. It specifies 12 operational components required for production readiness. All sprint mappings align with the existing 12-month build plan. Total schema additions: 8 new tables, 2 extended tables.